

Assignment II - SDD, BDD, and TDD in AI-Assisted Software Development

Student Information

- Name: 廖浩均
 - Student ID: 1121530
 - Course: CS351
 - Date: 2026/05/25
 - Due Date: 2026/05/31 23:59:59
-

1. Introduction

1.1 What AI-assisted software development is.

AI-assisted software development refers to the process in which engineers use generative artificial intelligence as a collaborative partner throughout the software development life cycle.

1.2 Why clear requirements are important when using AI tools.

The quality of AI-generated output depends heavily on the prompts provided by the user. When a prompt is vague or unclear, the AI may produce responses that contain hallucinations or inaccurate information. This may require users to spend additional time verifying the correctness of the results or engaging in multiple rounds of revision with the AI.

1.3 Why SDD, BDD, and TDD are useful in the AI era.

These three approaches provide AI with more precise prompts and structured thinking patterns, helping the AI follow a clearer direction and produce outputs that better satisfy the user's requirements.

2. Definition of SDD

2.1 Goal

The problem that the software is intended to solve.

2.2 Functional requirements

The functions that the software must provide.

2.3 Input and Output

The definition of how data enters and leaves the system. This describes what data the system receives and what results it must produce after processing.

2.4 Constraints

The rules that must be followed during development.

2.5 Acceptance criteria

A set of conditions used to check whether the system has been implemented correctly. Only when all conditions are satisfied can the project be considered successfully completed.

3. SDD: Student Grade Calculator

3.1 Goal

The primary objective of this module is to provide an automated engine that computes a student's overall academic standing for a semester. By substituting manual processing with systemized calculations, it reduces human grading errors and guarantees that final letter marks are assigned transparently and uniformly.

3.2 Functional Requirements

3.2.1 Grade Records Storage

The system shall allow instructors to save and store the calculated grades into a local database linked to the student's unique ID.

3.2.2 Score Statistics Tracking

The system shall automatically identify and highlight the highest and lowest scores among all the input assessment categories.

3.2.3 Performance Alert System

The system shall trigger a visual warning notification if a student's final weighted score falls below the passing threshold (60 points).

3.2.4 Report Exportation

The system shall provide an option to export the final score and letter grade breakdown into a printable PDF report.

3.3 Input

Student Unique ID

Assessment Scores

Instructor's Confirmation Command

3.4 Output

Database Confirmation Message

Statistical Highlights

Visual Warning/Alert

Printable PDF Report File

3.5 Grade Rules

3.5.1 Letter Grade Mapping Scale

The final calculated weighted score (S) determines the alphabetical mark assigned to the student profile: A (Excellent): $90.0 \leq S \leq 100.0$ B (Good): $80.0 \leq S < 90.0$ C (Satisfactory): $70.0 \leq S < 80.0$ D (Passing): $60.0 \leq S < 70.0$ F (Failing): $S < 60.0$

3.5.2 Performance Alert Trigger

Condition: If the final aggregated score $S < 60.0$, the system status switches to Failing. Action: The UI must instantly display a high-visibility warning notification (red highlight/pop-up) alongside the grade.

3.5.3 Database Record Linking Rule

When the instructor triggers the "Save" action, the system must validate that the Student Unique ID is not blank and matches standard institutional formatting before exporting the calculation results to the storage server.

3.5.4 Statistical Highlight Logic

The system compares the numerical arrays of all input assessment categories (e.g., homework, quizzes, midterm, final). It automatically flags the category with the highest value as Peak Performance and the lowest as Area for Improvement in the final output breakdown.

3.6 Acceptance Criteria

3.6.1 Storage Reliability

When the "Save" action is triggered with a valid student ID, the system must successfully write the data to the database, and the user interface must display a confirmation message: "Records saved successfully."

3.6.2 Statistical Highlight Accuracy

Given a set of input scores where Homework is 95 (highest) and Final Exam is 55 (lowest), the system output must explicitly and correctly flag Homework as the peak category and Final Exam as the area for improvement.

3.6.3 Alert System Trigger

If the final aggregated score is calculated to be 59.9 or lower, the system must instantly change the status indicator color to red and display a prominent text warning stating "Alert: Academic Performance Below Passing Threshold".

3.6.4 Document Integrity & Formatting

When the user executes the "Export PDF" command, the system must generate a downloadable .pdf file within 3 seconds. The exported document must completely and legibly contain the Student ID, individual score statistics, final weighted average, and the assigned letter grade without any missing fields.

4. Definition of BDD

Behavior-Driven Development (BDD) is an agile software development approach that describes software requirements from the perspective of user interactions and tangible business outcomes. It uses natural, human-readable language to define how a system should behave in concrete scenarios.

The framework relies on a structured, three-step formula known as the Given-When-Then format:

- Given: Establishes the predefined state or the initial context of the system.
- When: Identifies the specific trigger, action, or event performed by the user.
- Then: Asserts the expected reaction, outcome, or visible change produced by the system.

Why BDD is useful for communicating expected software behavior: Traditional software development often faces miscommunication between non-technical people (like PMs or owners) and developers. BDD solves this by using plain language and clear, real-world scenarios, making it much easier for PMs, QAs, and engineers to be on the same page regarding project requirements.

5. BDD: Student Grade Calculator

Scenario 1: System triggers red alert for failing grade and flags statistics

Given the continuous assessment mark is 50.0 And the mid-term examination mark is 45.0 And the end-of-term examination mark is 60.0 And the user provides a valid student ID "STU-2026" When the system calculates the student performance metrics Then the final aggregated score should be 51.5 And the system status should change to a red alert displaying "Alert: Academic Performance Below Passing Threshold" And the system should flag mid-term exam as the lowest assessment category

Scenario 2: System rejects out-of-bounds input and aborts calculation

Given the continuous assessment mark is 85.0 And the mid-term examination mark is 120.0 And the end-of-term examination mark is 75.0 When the instructor attempts to execute the final calculation Then the system must abort the calculation process completely And the system must block the generation of any letter grade or report And the system interface must display an error warning stating "Error: Evaluation inputs must be numeric values restricted to the 0-100 range."

6. Definition of TDD

Test-Driven Development (TDD) is an evolutionary software development technique where developers write automated test cases before writing any functional production code. Instead of building features first and testing them later, TDD reverses the traditional workflow to ensure code correctness and maintain high software quality from the very beginning.

The development process revolves around a continuous, micro-incremental feedback loop known as the TDD Cycle:

- **Red (Write a failing test):** The developer writes a specific unit test for a feature that does not exist yet. When executed, this test must fail (producing a "Red" status signal) because there is no implementation code to satisfy it.
 - **Green (Make the test pass):** The developer writes the bare minimum, simplest implementation code required to make the failing test pass successfully (producing a "Green" status signal). No extra or non-required code should be added at this stage.
 - **Refactor (Clean up the code):** Once the test turns green, the developer optimizes, structures, and cleans up the newly written code to eliminate duplication and improve readability. Crucially, this is done while running the tests continuously to ensure the implementation keeps passing and no bugs are introduced.
-

7.TDD: Student Grade Calculator

Scenario 1: Normal Test Cases

Test Case 1:Standard Passing Grade (B Grade)

Input

- Assignment: 85
- Midterm: 80
- Final Exam: 75
- Project: 0 (Note: This field is reserved for backward compatibility but carries 0% weight in this custom formula)

Expected Calculation

Final Score = $85 \times 0.40 + 80 \times 0.30 + 75 \times 0.30$ Final Score = $34.0 + 24.0 + 22.5 = 80.5$

Expected Output

- Final Score: 80.5
- Letter Grade: B

Test Case 2:Excellent Performance (A Grade)

Input

- Assignment: 92
- Midterm: 96

- Final Exam: 90
- Project: 0

Expected Calculation

Final Score = $92 \times 0.40 + 96 \times 0.30 + 90 \times 0.30$ Final Score = $36.8 + 28.8 + 27.0 = 92.6$

Expected Output

- Final Score: 92.6
- Letter Grade: A

Scenario 2: Boundary Test Cases**Test Case 1: Exact Minimum Passing Threshold (D Grade Boundary)****Input**

- Assignment: 60
- Midterm: 60
- Final Exam: 60
- Project: 0

Expected Calculation

Final Score = $60 \times 0.40 + 60 \times 0.30 + 60 \times 0.30$ Final Score = $24.0 + 18.0 + 18.0 = 60.0$

Expected Output

- Final Score: 60.0
- Letter Grade: D

Test Case 2: Just Below Passing Threshold (F Grade Boundary with Half-Up Rounding)**Input**

- Assignment: 59
- Midterm: 60
- Final Exam: 60
- Project: 0

Expected Calculation

Final Score = $59 \times 0.40 + 60 \times 0.30 + 60 \times 0.30$ Final Score = $23.6 + 18.0 + 18.0 = 59.6$ (Note: Per SDD protocol, 59.6 does not round up to 60.0, thus it remains a failing mark)

Expected Output

- Final Score: 59.6

- Letter Grade: F

Scenario 3: Invalid Input Test Cases

Test Case 1: Out-of-Bounds Score (Above 100)

Input

- Assignment: 105
- Midterm: 85
- Final Exam: 90
- Project: 0

Expected Calculation

The system detects that the Assignment score (105) exceeds the maximum allowed limit of 100.0. Calculation is instantly aborted.

Expected Output

- Final Score: Error
- Letter Grade: Error: Evaluation inputs must be numeric values restricted to the 0-100 range.

Test Case 2: Negative Score Input

Input

- Assignment: 75
- Midterm: -10
- Final Exam: 80
- Project: 0

Expected Calculation

The system detects that the Midterm score (-10) is below the minimum allowed limit of 0.0. Calculation is instantly aborted.

Expected Output

- Final Score: Error
- Letter Grade: Error: Evaluation inputs must be numeric values restricted to the 0-100 range.

8.Comparison of SDD, BDD, and TDD

Item	SDD	BDD	TDD
Full name	Specification-Driven Development	Behavior-Driven Development	Test-Driven Development

Item	SDD	BDD	TDD
Main focus	Architectural structure and technical boundaries	User interactions and business logic flow	Code implementation correctness and quality
Main question	<i>What structural components must be engineered?</i>	<i>How should the system react to user actions?</i>	<i>How do we programmatically validate the execution?</i>
Typical format	System blueprints, data schemas, API contracts	Given–When–Then conversational scenarios	Automated unit test cases and assertions
Primary Audience	System Architects, Project Managers, and Developers	Product Owners, QA Testers, and Non-technical Stakeholders	Software Developers and Automation Engineers
Development Stage	Initial planning and high-level architectural design	Requirements elicitation and acceptance criteria setup	Active coding phase (Micro-incremental feedback loops)
AI-era value	Serves as a structured prompt context , establishing rigid guardrails to eliminate architectural drift.	Acts as a behavioral prompting script , guiding AI to understand the precise intent of human-centric business logic.	Functions as an automated safety net , continuously verifying AI-generated output to catch hallucinations instantly.

9.Reflection

1.Which approach is easiest for you to understand: SDD, BDD, or TDD? Why?

- I often use AI as a copilot when I'm coding, and I have encountered many situations where I kept going back and forth with the AI but still could not get the result I wanted. This made me realize that if I can provide clearer and more structured information, such as requirements, design decisions, and expected behavior, the AI is more likely to generate useful results. In this sense, the concept of SDD makes sense to me because it helps define the system more clearly before implementation. If I have the opportunity, I would like to learn more about how SDD is used in real software development.

2.Which approach is most useful when working with AI coding tools? Why?

- I think SDD would be the most useful approach when working with AI coding tools. For me, using AI as a copilot means that I describe what I want to the AI, and the AI responds to my request. In this process, SDD can play an important role in making the communication smoother and more effective.

3.How can SDD help reduce unclear AI prompts?

- SDD can make this process smoother because it defines the requirements, system design, expected behavior, constraints, and other important details before implementation. With this structured information, AI coding tools can better understand the task and generate code or suggestions that are closer to what I actually need.

4.How can BDD help describe user expectations?

- BDD helps describe user expectations by using clear examples written from the user's perspective. It focuses on how the system should behave in specific situations, often using the format Given–When–Then. This makes requirements easier to understand for both developers and non-technical people, and helps ensure the final software matches what users actually need.

5.How can TDD help check whether AI-generated code is correct?

- TDD can help check whether AI-generated code is correct by requiring tests to be written before or alongside the code. After the AI generates code, developers can run these tests to see if the code meets the expected behavior. If the code passes the tests, it is more likely to be correct; if it fails, the test results show what needs to be fixed.

6.If you use AI tools in future software projects, how would you combine SDD, BDD, and TDD?

- I would combine SDD, BDD, and TDD as a structured workflow when using AI tools. First, I would use SDD to clearly define the system design, requirements, constraints, and expected features. Then, I would use BDD to describe user expectations with clear scenarios, such as Given–When–Then. Finally, I would use TDD to write tests and check whether the AI-generated code works correctly. Together, SDD helps guide the AI, BDD helps describe what users expect, and TDD helps verify the correctness of the code.

10.References / AI Tool Usage

- ChatGPT was used to help me translate my answers and check whether they were reasonable.
- Gemini was used to help me understand the three approaches.
- Reference website:
 - <https://agile3uncles.com/2025/10/08/sdd-bdd-tdd-kiro/>
 - <https://ithelp.ithome.com.tw/articles/10381105>